COMP 8051

Project Design Document

Group 11

Inez Yoon

A01066348

1. Team Members and Roles

Group Number	11
Number of Team Members	1
Member	Inez Yoon https://github.com/Inez-y
Project Repository	https://github.com/Inez-y/LastCarriage
Planning Tool	Figma https://www.figma.com/board/u4bMk49FUxSijOC4hs9Coy/TheLastCarriage?t=KW2a8yJLj4C90cqF-1 Trello https://trello.com/invite/b/69ae4e42b04185dc6fe525f1/ATTI4505c44f8181bb73f863d1016df4e13dAFCAEEFC/the-last-carriage

Responsibilities

As this project will be completed individually, I will be responsible for all aspects of development, including both technical implementation and project documentation.

Programming Responsibilities	Documentation Responsibilities:
● Game engine architecture design ● Implementation of the entity-component system ● Rendering system and asset management ● Input handling ● Collision detection and physics interactions ● Scene and map management ● User interface and HUD implementation ● Game state and progression management	● Project design documentation ● Technical architecture documentation ● System-level documentation and comments ● Development progress tracking ● Final project report and presentation materials ● Repository documentation and setup instructions

2. Engine Design

Overview

LastCarriage is built on a custom 2D game engine written in C++ using SDL. The engine was designed to support a side-scrolling action game with stage progression, combat, item collection, enemy behaviors, and UI rendering. Rather than relying on a monolithic game class, the project separates engine-level infrastructure from game-specific logic through an Entity-Component-System-inspired architecture, specialized gameplay systems, factory-based entity creation, and a data-driven map pipeline.

The overall design goal is to keep the engine modular, readable, and extensible. Core runtime responsibilities such as map loading, stage transitions, rendering, movement, combat, and interaction are handled by separate classes with clearly defined purposes. Map content such as terrain, spawn points, enemies, items, colliders, and doors is loaded from external TMX files, allowing gameplay content to be changed without rewriting the core engine.

Multimedia Library

The engine uses SDL as its primary multimedia framework. SDL provides the low-level runtime support needed for:

- window creation and management
- hardware-accelerated rendering
- keyboard input processing
- timing and frame-based execution

Additional SDL libraries are used to extend functionality:

- SDL_image for texture loading
- SDL_ttf for font and UI text rendering

The current implementation also includes support for UI font loading and texture-based tile rendering during engine initialization.

Engine Type

The engine is a 2D sprite- and tilemap-based engine intended for side-scrolling gameplay. The world is rendered using a tilemap for environment layout and sprite-based entities for the player, enemies, items, and interactive objects. The tilemap renderer draws visible terrain from map data, while gameplay entities are managed through the ECS layer and rendered separately.

This approach supports a clear separation between static world data and dynamic gameplay objects, while remaining practical for a solo project with a manageable scope.

Architecture Approach

The engine follows a hybrid ECS architecture.

In this design:

- Entities represent game objects
- Components store the data associated with those objects
- Systems process entities with specific component combinations
- Managers and helper classes outside the ECS handle broader runtime coordination, such as stage loading, map parsing, and shared game state

This hybrid approach was chosen to balance architectural clarity with implementation practicality. The ECS layer handles gameplay composition, while higher-level runtime classes coordinate stage transitions, map loading, rendering resources, and shared state. For example, the engine uses a `GameContext` structure to share runtime dependencies such as the renderer, active map, player reference, UI font, and pending stage changes across multiple systems.

This architecture also separates map parsing from stage spawning. `MapLoader` reads TMX files into a plain data structure, `MapData` stores the parsed content, `Map` provides runtime access and tile rendering, and `StageManager` converts the loaded data into active ECS entities such as doors, enemies, and items. This separation improves modularity and makes the engine easier to debug and extend.

Engine Design Goals

The `Game` class is responsible for initializing SDL, creating the window and renderer, loading the UI font and tileset texture, starting the initial stage, and running the main loop. During each frame, it processes input, updates systems, applies stage transitions when requested, and renders the current map and entities.

The engine currently coordinates gameplay through dedicated systems, including:

- movement
- combat
- item interaction
- door interaction
- rendering

These systems are owned by the main game runtime and updated in a structured order each frame.

Map and Stage Design

One of the strongest architectural parts of the engine is its data-driven stage system.

Map files define:

- terrain layout
- named spawn points
- enemy spawn locations
- item spawn locations
- collision regions
- doors that connect stages

The loader identifies object groups by name rather than relying on their order in the XML file. This makes the map system more robust and extensible when additional map layers are introduced. Parsed data is stored in `MapData`, and `StageManager` uses that data to clear old stage entities and spawn the next set of gameplay objects while preserving or repositioning the player.

This design reduces hardcoding and allows level content to be adjusted in Tiled without significant engine changes.

Entity Creation Strategy

The engine uses factory classes to create common gameplay entity types. Separate factories exist for players, enemies, items, bullets, and doors. This keeps entity setup centralized and consistent, which improves maintainability and reduces duplication across systems and scenes. The current stage loader, for example, uses factories to spawn doors, enemies, and items from loaded map data.

The enemy creation pipeline also supports multiple enemy types through type-based configuration, allowing the engine to scale behavior and stats without rewriting the full entity creation process.

Engine Design Goals

The implemented engine is designed to support the following capabilities:

- modular entity and component management
- reusable system-driven gameplay logic
- 2D tilemap and sprite rendering
- player input and movement handling
- combat, ammo, and projectile spawning
- enemy spawning and AI state support
- collision and interaction logic
- stage transitions through doors
- item pickup handling
- UI rendering through SDL_ttf
- shared runtime coordination through a central game context

3. Systems

The engine is organized around a set of focused gameplay systems. Instead of placing all logic inside a single game class, the project separates update responsibilities into dedicated systems for movement, combat, interaction, rendering, and stage management. The main runtime coordinates these systems each frame, while each system handles one specific aspect of gameplay behavior. This improves modularity, readability, and extensibility.

System Architecture Overview

At runtime, the `Game` class initializes SDL, creates the renderer and shared resources, loads the first stage, and then runs the main loop. During each frame, the engine performs input handling, updates gameplay systems, processes stage transitions if needed, and renders the current world state.

The project currently uses the following main systems and runtime coordinators:

- movement system
- combat system
- item system
- door system
- render system
- stage manager
- map loader and runtime map support

Together, these systems form the core gameplay pipeline of the engine.

Core Game Loop System

The `Game` class functions as the top-level runtime controller. It is responsible for:

- SDL initialization and shutdown
- window and renderer creation
- loading shared resources such as the tileset texture and UI font
- starting the initial stage
- running the main loop
- forwarding update work to gameplay systems
- rendering the world each frame

This class does not directly implement every gameplay feature. Instead, it acts as the orchestrator that connects the ECS manager, map, camera, shared game context, and major systems. That separation is important because it keeps the runtime structured and easier to maintain.

Entity and Component Management

The engine uses a custom ECS structure in which entities are lightweight containers and components hold the data needed by systems. Systems then operate on entities that contain the required component sets. This allows gameplay objects such as the player, enemies, items, bullets, and doors to be assembled from reusable pieces rather than fixed inheritance trees.

This design supports flexibility and keeps behavior modular. For example, stage loading does not hardcode all object logic directly into the map system. Instead, entities are spawned from data and then updated through systems based on their attached components. That makes it easier to add new gameplay objects later without rewriting the engine structure.

Transform and Movement System

The movement system updates entity movement and is one of the core systems in the frame update pipeline. It is responsible for applying movement-related behavior using entity data and the shared runtime context. This includes player movement and other movement-driven entities such as enemies. The project also separates enemy-specific movement logic into its own movement-related logic unit, showing a more modular design than a single all-purpose update function. The movement system is updated every frame before combat and interaction processing.

Movement in the engine is supported by data components such as transforms, physics-related data, and input or AI state, depending on the entity type. This separation allows player and enemy motion to be processed within the same broader architecture while still supporting different behaviors.

Combat System

The combat system handles combat-related gameplay logic. This includes processing bullet spawning requests, weapon-related state, attacks, and damage interactions. The shared game context includes weapon cooldown timing and pending bullet spawn requests, which allows combat events generated during input handling to be processed in a structured way during the update phase instead of immediately mixing them into unrelated code.

The combat system is a central part of the current gameplay loop because it connects player input, weapon state, projectile creation, and enemy interactions. By keeping combat in its own system, the code remains cleaner and easier to extend with additional attack types or enemy behaviors later.

Item Interaction System

The item system is responsible for handling item pickups and other item-related gameplay interactions. Items are map-driven and are spawned into the world by the stage manager using item spawn data loaded from the current TMX map. This means item placement is authored externally in the map rather than hardcoded into the game loop.

This system supports the gameplay loop by allowing the player to collect resources and interact with the level in a way that demonstrates the usefulness of the ECS and map-driven design.

Door and Stage Transition System

The door system manages transitions between stages. Door locations and destination information are loaded from the map file, stored as door data, spawned as ECS entities, and then processed by the door system during gameplay. The shared game context stores whether a stage change has been requested and which map and spawn point should be loaded next.

This is one of the strongest examples of interlinked engine architecture in the project:

- the map file defines the doors
- the stage manager spawns them
- the door system detects their use
- the game runtime performs the actual stage transition
- the next stage is loaded and the player is repositioned

Because of this pipeline, stage progression is data-driven and extensible rather than hardcoded room by room.

Rendering System

The render system is responsible for drawing gameplay entities and UI elements. Tile-based world rendering is handled by the `Map` runtime object, which renders the terrain layer based on loaded tile data, tile dimensions, and the current camera offset. Entity and UI rendering are then handled separately by the render system, producing a clear division between map rendering and dynamic object rendering.

The project also includes runtime support for a UI font in the shared context, showing that rendering is not limited to sprites alone. This supports HUD-style feedback such as gameplay information displayed to the player.

Animation Support

Animation is treated as a distinct part of the engine architecture rather than being merged entirely into sprite rendering. The project includes an animation system and animation-related components, allowing visual states such as idle, movement, and attack animations to be managed more systematically. This is an important architectural improvement because it separates gameplay state from rendering state and makes sprite behavior easier to organize.

Animation support is especially important for a character-based action game because it connects movement, combat, and visual feedback across multiple entity types.

Collision and Physics Support

The current architecture also includes collision and physics support as part of the engine's system structure. Collision regions are loaded directly from the map into structured map data, which allows the world to define physical boundaries externally in Tiled. These collision regions are then available for gameplay logic and movement constraints during runtime.

Enemy and player entities also use movement- and physics-related components as part of their setup, showing that the engine separates collision and motion concerns into reusable data rather than hardcoding them into single-purpose classes. Enemy creation, for example, includes transform, sprite, physics, and health.

Map Loading System

The map loading system is an important architectural layer because it separates content parsing from gameplay runtime behavior. `MapLoader` reads TMX files and extracts:

- terrain tiles
- named spawn points
- enemy spawns
- item spawns
- collision regions
- doors

It identifies object groups by layer name rather than depending on XML order, which makes the engine more robust and easier to extend. The parsed results are stored in `MapData`, then used by `Map` and `StageManager` during runtime.

This makes the level system strongly data-driven and is one of the clearest examples of good system separation in the project.

Stage Management System

The stage manager functions as the bridge between map data and active gameplay. When a stage is loaded, it:

- clears old non-player entities
- clears pending bullet requests
- activates the newly loaded map
- spawns doors
- spawns or repositions the player
- spawns enemies
- spawns items

This makes stage loading much more than simple file parsing. It is a full runtime system that translates data into active ECS content. The player is preserved across stage loads, while other stage-specific objects are rebuilt based on the newly loaded map.

This architecture is particularly useful in a multi-stage game because it keeps stage changes consistent and avoids duplicating spawn logic throughout the codebase.

Shared Runtime Context

A shared `GameContext` structure is used to connect systems that need access to runtime-wide information. It stores references and values such as:

- renderer
- tileset texture
- UI font
- camera
- active map
- player entity
- stage transition requests
- cooldown timers
- coin count
- pending bullet spawns

This reduces unnecessary global coupling while still allowing systems to coordinate through a controlled shared interface. It is especially useful for linking otherwise separate systems such as combat, doors, rendering, and stage management.

Input Handling

Input is handled in the runtime loop and connected to ECS-driven gameplay through components and systems. The `Game` class processes SDL events and reads keyboard state, while player-related input data is passed into the player's input component and combat state. This design keeps low-level input collection in the runtime layer while allowing gameplay interpretation to remain component- and system-based.

This is another example of the hybrid architecture approach: SDL-specific input stays at the runtime boundary, while gameplay consequences are distributed into the ECS structure.

System Design Summary

The final system architecture emphasizes separation of concerns and practical ECS-based design. Instead of treating the engine as a single large block of update code, the project divides responsibilities across systems for movement, combat, interaction, rendering, animation, collision support, map loading, and stage transitions. This improves maintainability and also makes the architecture easier to explain in an academic setting.

Overall, the system design supports the main goals of the project:

- modular engine structure
- data-driven gameplay content
- clear division between engine and game logic
- extensibility for future features
- maintainable solo-project development workflow

4. Game Design

Game Title

Last Carriage

Genre

2D Action / Adventure

The game combines side-scrolling movement, combat, item collection, and stage progression. While the original concept included stronger narrative and strategy elements, the current implementation is best described as a 2D action game with environmental progression and dystopian worldbuilding.

Game Concept

LastCarriage takes place in a dystopian future where the outside world has become uninhabitable. Humanity survives aboard a massive train that must continue moving in order to sustain life. Society on the train is divided by class, with the harshest conditions concentrated in the rear carriages and increasing privilege toward the front. The player begins in the back section and progresses through different areas of the train, confronting hostile forces and surviving increasingly dangerous spaces while moving forward.

In gameplay terms, this concept is expressed through a series of connected stages. The player explores train sections, fights enemies, collects useful items, and uses doors to move from one area to the next. This stage-based structure is directly supported by the engine's map and stage management pipeline, where maps define enemy spawns, item spawns, collision regions, named spawn points, and door-based transitions.

Multiplayer

The game will be single-player only. Multiplayer functionality will not be included due to the increased technical complexity and the limited scope of a solo project.

Core Gameplay Loop

The current game loop is built around:

1. entering a stage
2. navigating the environment
3. fighting enemies
4. collecting items or resources
5. surviving hazards and managing health/ammo
6. using doors to progress into the next section

This structure is supported by the current runtime design. The game initializes a stage, updates movement and combat, processes items and doors, and loads the next map when a stage transition is requested.

Gameplay Features

The implemented and intended gameplay features include:

- player movement and exploration
- ranged combat using a weapon and ammo system
- reload management
- enemy encounters
- item pickups
- door-based stage transitions
- heads-up display elements
- multi-stage progression across connected map areas

The game runtime already includes weapon cooldowns, bullet spawn requests, coin tracking, item handling, and stage transition requests through the shared context and system update flow.

Enemy encounters are also designed to support more than one enemy type. The current enemy creation pipeline includes type-based setup for patrol, shooter, flying, and boss-style enemies, which helps the game demonstrate engine flexibility as well as gameplay variety.

Target Platform

Primary target platform:

- Desktop PC (Mac)

Due to SDL2's cross-platform capabilities, the engine may potentially support additional platforms such as Windows if time permits.

Player Experience

The intended player experience is one of steady forward progression through hostile train environments. The player should feel that each carriage is part of a larger connected world, with movement through doors representing advancement through the train. Combat and pickups are used to create tension and reward, while the stage structure supports pacing and variation across different sections.

From a design perspective, the game aims to present:

- immediate action through responsive movement and shooting
- short-term goals through survival and item collection
- medium-term goals through reaching the next carriage
- broader thematic progression through movement from the rear to the front of the train

World and Level Design

The game world is structured as a set of connected 2D stages built with tilemaps. Each stage can contain:

- terrain tiles
- player spawn points
- enemy placements
- item placements
- collision boundaries
- doors that lead to other maps

This design allows each section of the train to be authored externally in Tiled and loaded at runtime through the engine's map system. The map loader parses this data into structured map content, and the stage manager uses that data to spawn the active gameplay entities.

Because of this, the game's level design is strongly data-driven. New rooms, enemy placements, and transitions can be added without changing the core engine structure

Combat Design

Combat is currently centered around player shooting, ammo usage, and enemy interaction. The player can fire projectiles, consume ammo, and reload, while combat-related actions are processed through the engine's combat flow rather than being hardcoded directly into rendering or map logic. The game loop collects input, creates pending bullet spawn requests, and then processes those through the update pipeline.

This makes combat one of the clearest examples of interlinked system design in the project:

- input triggers an attack request
- weapon state controls whether shooting is allowed
- the combat pipeline processes the shot
- bullets and damage interact with the game world and enemies

This also helps the final game showcase the strengths of the engine architecture.

Enemy Design

Enemies are intended to provide both gameplay challenge and architectural variety. The game supports multiple enemy categories through the factory pipeline, including patrol, shooter, flying, and boss configurations. Different enemy types can vary in movement, aggression range, attack range, and durability.

This supports two important goals:

- creating more interesting gameplay encounters
- demonstrating that the engine can support multiple gameplay archetypes through reusable systems and components

Item and Progression Design

Items are used to reinforce exploration and reward the player for moving through each stage. Item spawns are defined in the map rather than hardcoded in the main runtime. This means progression-related resources can be placed differently from stage to stage, allowing the pacing of the game to be tuned through level design.

The current runtime context also tracks coin count, which supports collectible-based progression or reward systems.

User Interface

The game includes HUD-style feedback to communicate important player information during play. The runtime supports UI font loading and rendering, which enables on-screen display of player-related information such as health, ammo, and other gameplay values.

The user interface is designed to support clarity rather than visual overload. Since the project is focused on architecture as well as gameplay, the HUD is intended to reinforce game state visibility while remaining lightweight.

Art and Asset

The game uses a 2D sprite- and tile-based visual style. The train environment is represented through tilemaps, while characters, enemies, items, and interactive gameplay elements are represented with sprites. This matches the engine's rendering model and keeps asset production manageable for a solo project.

The visual design is intended to support environmental storytelling across train sections, with each area feeling like a distinct carriage or zone. This supports the overall theme of moving through a socially stratified train while also providing natural structure for stage-based gameplay.

Game Design Summary

The final game design for *LastCarriage* is centered on a playable 2D action experience that showcases the engine's architecture through:

- map-driven stage design
- ECS-based gameplay objects
- system-driven combat and interaction
- door-based progression
- reusable enemy and item spawning
- HUD-supported player feedback

Rather than trying to implement every part of the original narrative proposal at full scale, the current design focuses on delivering a stable, coherent, and polished gameplay slice that demonstrates both engine capability and game implementation quality.

5. Tools and Risks

Development Tools

The development of LastCarriage uses a focused toolchain centered on C++, SDL, and external content tools suitable for a solo game architecture project.

Programming and Engine Development

The main programming language for the project is C++. The engine and game are developed in a modular source structure that separates engine code, gameplay systems, map logic, factories, and components. The runtime currently uses SDL3, along with SDL3_image for texture loading and SDL3_ttf for UI text rendering. The game initialization flow creates the SDL window and renderer, loads the UI font, and loads the tileset texture during startup.

Build configuration is managed through CMake, and the project is developed in a desktop environment using CLion. Version control is handled through Git which supports iterative development and cleaner project tracking. The original project plan also identified CLion, Git, and GitHub as part of the development workflow.

Map and Content Tools

The game uses Tiled for map creation. This is an important part of the workflow because the engine loads TMX map files and parses level content such as terrain, spawn points, enemy spawns, item spawns, collision regions, and door transitions from those files. This allows the level design process to remain data-driven rather than hardcoded into the engine.

For visual content, 2D sprite and tileset assets are used for characters, environments, and gameplay objects. The earlier project plan identified tools such as Aseprite or Piskel for sprite creation and iteration, which remain appropriate for this style of project.

Documentation and Planning

Project documentation is supported through the repository README, technical writeups, architecture diagrams, comments, and presentation materials. This is especially important in a game architecture course because the project is evaluated not only as a playable game, but also as an example of system design, code organization, and engineering clarity. The original project outline also emphasized documentation, progress tracking, and final presentation materials as a major responsibility area.

Risks

Because *LastCarriage* combines both engine development and game development in a single academic project, there are several technical and production risks that affect the final result.

1. Scope Management

The largest risk in the project is scope. Building a custom engine and a complete game at the same time can easily lead to overexpansion. Systems such as ECS architecture, map loading, rendering, combat, UI, item handling, and stage transitions all require time to design, debug, and polish. The earlier planning document already identified scope control as one of the main development risks.

Mitigation:
The project addresses this by prioritizing a stable playable core first. The main focus is on a complete gameplay slice with movement, combat, item interaction, map-driven spawning, and stage progression. Additional or more ambitious features should only be expanded after the main loop is stable and presentable.

2. Engine Architecture Complexity

A custom engine introduces architectural risk, especially when using a hybrid ECS structure. If system responsibilities are unclear, logic can become too centralized or too tightly coupled. This risk is especially relevant when multiple systems depend on shared runtime state such as the active map, player entity, renderer, stage transitions, and combat events. The project manages this through a dedicated `GameContext` structure and a staged pipeline that separates map parsing, runtime map access, stage spawning, and gameplay system updates.

Mitigation:
Keep each class focused on a small responsibility. For example:

- `MapLoader` handles parsing
- `MapData` stores loaded content
- `Map` handles runtime access and tile rendering
- `StageManager` turns map data into active entities
- gameplay systems handle movement, combat, interaction, and rendering

3. Runtime Stability and Debugging

Another major risk is runtime instability. In an ECS-based game, bugs related to destroyed entities, missing components, incorrect stage transitions, repeated spawns, animation issues, and timing errors can be difficult to trace. Since the game loop updates multiple systems in sequence and supports dynamic stage changes, debugging requires careful control over entity lifetime and update order. The current runtime already uses an ordered update loop and refresh calls to reduce stale entity problems, but this remains a critical risk area.

Mitigation:
Test systems incrementally and verify one subsystem at a time. Stage loading, combat, movement, rendering, and UI should each be tested independently before full integration. A small number of polished, reliable mechanics is more valuable than a larger number of unstable ones.

4. Content and Asset Production

As a solo project, creating enough maps, sprites, UI elements, and polish assets can become time-consuming. The original plan already noted that asset production could become a limitation and suggested using placeholder or openly licensed assets when necessary.

Mitigation:
Use a practical visual scope. The project already benefits from a 2D sprite and tilemap approach, which is more manageable than a large-scale art pipeline. Placeholder assets can continue to support gameplay testing, while final asset polish can be limited to the most visible parts of the game.

5. Solo Development Constraints

All programming, debugging, design, integration, documentation, and presentation work must be completed by one developer. This limits how much content and polish can realistically be completed within the course timeline. The original planning document recognized solo workload as a significant constraint.

Mitigation:
The project should prioritize:

- a stable game loop
- a clear architectural structure
- a small number of polished stages
- readable documentation
- a strong final presentation

This keeps the project aligned with the grading rubric, which values architecture quality, implementation completeness, and code stability.

Project Management Risks

1. Time Allocation Between Engine and Game

A common risk in architecture projects is spending too much time on engine refactoring and not enough on gameplay completeness, or the reverse. Since this project is graded on both engine architecture and game realization, imbalance in either direction can hurt the final result.

Mitigation:
Development should be split into phases:

1. core engine systems
2. playable game loop
3. polish and bug fixing
4. documentation and presentation preparation

This ensures that the project remains both technically strong and playable.

2. Documentation Quality

Even a well-structured codebase can lose marks if the architecture is not explained clearly. Since the project includes a design document, README, diagrams, and a final presentation, weak documentation is a real risk.

Mitigation:
Maintain concise technical documentation that explains:

- the ECS structure
- the map loading pipeline
- the stage transition pipeline
- the role of systems and factories
- the gameplay loop supported by the engine

This helps the final submission communicate design quality clearly.

Conclusion

In conclusion, *LastCarriage* demonstrates the design and development of a custom 2D game engine and game built around a modular, hybrid ECS architecture. The project emphasizes clear system separation, data-driven stage design, and maintainable code organization while supporting core gameplay features such as movement, combat, interaction, rendering, and progression. By balancing technical architecture with a playable game implementation, the project shows how a focused engine can be designed to support both extensibility and a complete gameplay experience within the scope of a solo academic project.